

SIMATS ENGINEERING ASSESSMENT TOOL 1

Course: Computer Architecture (CSA12) | CO1 | Total Marks: 25

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4) Assessment Tool Weightage: 40%

ANNEXURE A APPLICATION BASED QUESTIONS

QUESTIONS:5

TOTAL MARK:25

ANNEXURE A APPLICATION BASED QUESTIONS

I **RITHISH KUMAR.D(192565089)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A handwritten signature in blue ink, appearing to read 'Rithish Kumar.D', with a horizontal line extending to the right.

Question 1: CPU–Memory–I/O Interaction and Bus Structures in a Smart City Surveillance System

***Given Scenario:** A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.*

1. Understanding the Problem Context

A smart city surveillance network is essentially a high-throughput, real-time data-processing pipeline. Multiple IP cameras act as I/O devices continuously streaming compressed video frames that must be moved into main memory, analyzed by the CPU (or a co-processor running detection algorithms), and, when a threat is identified, acted upon within a strict time budget. The reported "lag" during peak hours is a classic symptom of the system's data path becoming saturated faster than the CPU can consume and process the incoming data — i.e., the problem is not purely computational, it is architectural, rooted in how the CPU, memory, and I/O subsystem communicate.

2. How CPU–Memory–I/O Interaction Causes the Bottleneck

In a von Neumann-style architecture, the CPU, memory, and I/O controllers all share a common communication path. Each camera feed generates interrupts or DMA (Direct Memory Access) requests to transfer frame buffers into RAM. As the number of active cameras increases:

- I/O contention: Multiple I/O devices compete for the same bus/interface to write data into memory, causing queuing delays.
- Memory bandwidth saturation: Frame buffers occupy memory bandwidth needed simultaneously for CPU instruction fetches and data reads, creating a bus contention problem.
- Interrupt overhead: A high rate of interrupts from cameras forces frequent context switches, increasing the effective cycles-per-instruction (CPI) for the detection algorithm.
- CPU stall cycles: If data has not yet arrived from memory (due to bus congestion), the CPU stalls, wasting clock cycles that should have been used for threat-detection computation.

This chain reaction — I/O contention → memory bandwidth saturation → CPU stalls — is the direct architectural cause of the delayed threat detection response during peak load.

3. Bus Structures: Single-Bus vs Multi-Bus Architecture

Single-Bus Architecture: In a single-bus system, the CPU, memory, and all I/O devices share one common set of address, data, and control lines. Only one transfer can occur at any instant. When camera feeds increase, every image-frame transfer, every CPU-memory instruction fetch, and every actuator/alert signal must queue for the same physical bus. This design is simple and low-cost, but it becomes the primary bottleneck under the high, simultaneous data-transfer load of a surveillance network — the very lag observed in the scenario.

Multi-Bus Architecture: A multi-bus (or multiple-bus) system separates traffic onto dedicated paths — for example, a high-speed memory bus between CPU and RAM, a separate I/O bus for camera/peripheral traffic, and a system bus for control signals. This allows video data streaming in from cameras (via DMA on the I/O bus) to happen concurrently with the CPU fetching instructions and reading partially processed frames from memory (via the memory bus). Because transfers no longer compete for a single shared resource, effective throughput increases substantially and CPU stall time is reduced.

For a surveillance system, adopting a multi-bus (or hierarchical bus, e.g., a local processor bus + backplane bus + expansion bus) architecture directly addresses the peak-hour lag by allowing parallel data movement instead of serialized contention.

4. Improving the Instruction Execution Cycle

Beyond bus redesign, performance can be improved by optimizing the fetch–decode–execute cycle that the CPU runs for every detection instruction:

- **Pipelining:** Overlapping the fetch, decode, and execute stages of successive instructions (e.g., processing pixel-comparison instructions for frame N while fetching instructions for frame N+1) increases instruction throughput without increasing clock speed.
- **DMA-based I/O:** Allowing camera controllers to write frames directly into memory via DMA, bypassing the CPU for the transfer itself, frees CPU cycles previously spent executing I/O-handling instructions.
- **Cache hierarchy:** Keeping frequently accessed detection-model weights and recent frame data in L1/L2 cache reduces the number of instruction-cycle stalls caused by waiting on main memory.
- **Interrupt prioritization / interrupt coalescing:** Batching multiple camera interrupts instead of servicing each individually reduces the interrupt-handling overhead per execution cycle.

5. Recommended Solution and Conclusion

The peak-hour lag in the surveillance system results from the combined effect of shared-bus contention and CPU stalls caused by memory/I/O congestion. Migrating from a single-bus to a multi-bus (or bus-hierarchy) architecture separates camera I/O traffic from CPU–memory traffic, eliminating the primary contention point. Combined with pipelined instruction execution, DMA-driven I/O, and an efficient cache hierarchy, the effective CPI for the detection workload drops significantly, clock cycles are used more productively, and the system can sustain real-time threat detection even under peak camera load. This demonstrates that performance in data-intensive embedded systems is governed as much by architectural bus design as by raw processor speed.

Simulate cameras sending data to CPU through memory.

CODE:

```
#include <stdio.h>
```

```
int main() {
```

```
    int cameraData[4] = {120, 135, 140, 160};
```

```
int memory[4];

// I/O to Memory
for(int i=0;i<4;i++)
    memory[i] = cameraData[i];

// CPU Processing
for(int i=0;i<4;i++) {
    if(memory[i] > 150)
        printf("Threat Detected at Camera %d\n", i+1);
    else
        printf("Camera %d Normal\n", i+1);
}

return 0;
}
```

Application: Simulates camera → memory → CPU processing.

Question 2: Reducing CPI and Improving CPU Performance in a Real-Time Stock Trading System

***Given Scenario:** A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode–execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.*

1. Understanding the Problem Context

A stock trading engine must validate incoming orders, check account balances, compare bid/ask prices, and execute trades — all under microsecond-level latency requirements. Even though the processor executes millions of instructions per second, the reported transaction delays under high load indicate that the effective execution time per instruction is higher than expected. Since $\text{Execution Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$, and instruction count and clock cycle time are largely fixed by the workload and hardware clock, the delay must be driven primarily by an elevated CPI (Cycles Per Instruction) — exactly as stated in the scenario.

2. Impact of the Fetch–Decode–Execute Cycle on Execution Time

Every trading instruction — comparing prices, updating order books, branching on condition checks (e.g., "if price $\leq$ limit, execute trade") — passes through the fetch, decode, and execute stages. In a naive (non-pipelined) implementation, these three stages execute strictly sequentially for each instruction, so the CPU cannot begin fetching the next instruction until the current one fully completes. Two workload characteristics of this scenario make that especially costly:

- Frequent memory access: Order-book lookups and account-balance checks require reading/writing memory. If data is not in cache, the execute stage stalls waiting for a

memory response, directly inflating CPI (a cache miss can cost tens to hundreds of cycles instead of one).

- Frequent branch instructions: Conditional trade-execution logic (buy/sell/hold decisions) causes frequent branches. Each branch disrupts sequential instruction fetching — the CPU may fetch the wrong path and must flush and refetch, adding "branch penalty" cycles to CPI.

Because CPI directly multiplies execution time, even a small increase in average CPI (say from 1.2 to 2.5) can nearly double the time needed to process the same number of trading instructions, producing exactly the user-visible transaction delay described.

3. Root Causes of High CPI in This Scenario

- Memory-access latency: RAM access is orders of magnitude slower than register/cache access; frequent order-book reads without adequate caching cause repeated stalls.
- Branch misprediction: Without prediction logic, every conditional trade check may cause a pipeline flush.
- Resource/data hazards: Successive instructions depending on results not yet available (e.g., using a just-computed price before it is written back) force pipeline stalls.
- Lack of instruction-level parallelism exploitation, forcing effectively serial execution under load.
-

4. Architectural Enhancements to Reduce CPI

a) Instruction Pipelining: Splitting execution into overlapping fetch/decode/execute (and further into fetch, decode, execute, memory-access, write-back) stages allows multiple instructions to be in different stages simultaneously, improving instruction throughput and lowering effective CPI, provided hazards are managed.

b) Branch Prediction and Speculative Execution: Adding a branch predictor (static or dynamic, e.g., a branch-history table) allows the CPU to guess the outcome of buy/sell conditionals and continue fetching along the predicted path, sharply reducing the branch penalty that inflates CPI in decision-heavy trading logic.

c) Multi-level Cache Hierarchy: Introducing/optimizing L1/L2/L3 caches for order-book and account data reduces the frequency and cost of memory stalls, since hot trading data (actively traded symbols) stays close to the CPU.

d) Pipeline Hazard Handling: Techniques such as forwarding/bypassing (feeding a computed result directly to the next instruction instead of waiting for write-back) and out-of-order execution reduce data-hazard stalls.

e) Superscalar / Multiple Issue: Issuing more than one instruction per cycle (where independent instructions exist, e.g., validating two unrelated orders) further reduces average CPI below 1.

5. Conclusion

The stock trading system's high CPI stems directly from memory-access stalls and branch penalties occurring within the fetch–decode–execute cycle. Because execution time scales linearly with CPI, reducing it through pipelining, branch prediction, an optimized cache hierarchy, and hazard-mitigation techniques such as forwarding and out-of-order execution will proportionally reduce transaction latency, even under high market-load conditions. This confirms that in latency-critical applications, architectural CPI-reduction techniques are as important as raw clock frequency for meeting real-time performance requirements.

Simple trading decision using branch instructions.

CODE:

```
#include <stdio.h>

int main() {

    int marketPrice = 950;

    int targetPrice = 1000;

    if(marketPrice <= targetPrice)

        printf("Buy Stock\n");

    else

        printf("Wait\n");

    return 0;}
```


Question 3: Addressing Modes and Reliable Instruction Design in an 8085-Based Industrial Temperature Control Unit

***Given Scenario:** An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.*

1. Understanding the Problem Context

In an 8085-based temperature control unit, a sensor (via an ADC and input port) periodically supplies a digital temperature value that the microprocessor reads, compares against a threshold, and uses to drive an actuator (heater/cooler relay) through an output port. The reliability of this control loop depends entirely on the CPU fetching and storing the correct data from the correct memory/register location on every cycle. The reported issue — occasionally incorrect temperature readings being processed — points to a data-access fault, which in 8085 assembly programming is most often traced to the misuse of addressing modes rather than a hardware sensor fault.

2. How Different Addressing Modes Influence Data Access

Immediate Addressing: The operand (a constant, e.g., a threshold temperature value) is specified directly in the instruction, e.g., MVI A, 32H. This is fast and reliable for fixed reference values, but if the threshold changes conditions, hard-coded immediate values can become stale/incorrect.

Register Addressing: Data is accessed from CPU registers, e.g., MOV A, B. Fast, but only holds a value temporarily — if the sensor reading is overwritten by another operation using the same register before it is processed, the wrong value is used.

Direct Addressing: The instruction specifies the exact memory address of the data, e.g., LDA 2050H. This is used to read the latest stored sensor value from a fixed memory location. An incorrect or mistyped address constant here will silently read stale or unrelated memory, which is a very common source of the "incorrect readings" described in the scenario.

Indirect Addressing (Register Indirect): The address of the data is held in a register pair (e.g., LDAX B, or using H-L as a pointer, MOV A, M). This is used when the sensor buffer location

can change (e.g., cycling through multiple sensor channels). If the pointer register pair (H-L) is not correctly updated/reinitialized between reads, the CPU will read from the wrong memory location — a very likely cause of intermittent incorrect readings in a multi-sensor system.

Implied/Implicit Addressing: Instructions such as CMA or RAL operate on an implied register (accumulator); no explicit operand is needed, but the programmer must be certain of what value currently resides there.

3. Possible Causes of the Reported Errors

- Using direct addressing with an incorrectly specified or un-updated memory address, causing the CPU to read outdated or wrong sensor data.
- Failing to reinitialize the H-L register pair before an indirect read when cycling between multiple sensor channels, causing cross-channel data contamination.
- Reusing a register (e.g., accumulator A) for multiple purposes between the sensor-read and actuator-write instructions without saving/restoring its value, so a later instruction overwrites the temperature value before it is compared.
- Hard-coded immediate threshold values that do not match the calibrated sensor range, leading to logically "correct" but practically wrong actuator decisions.
- Timing/synchronization errors where the CPU reads the input port before the ADC conversion has completed, effectively an addressing-and-timing combined fault at the I/O port address.

4. Using the 8085 Instruction Set for Accurate and Reliable Operation

- Use IN and OUT instructions with clearly defined, verified port addresses for sensor input and actuator output, and always confirm ADC conversion completion (via a status/ready line or delay routine) before executing IN.
- Prefer register-indirect addressing (H-L pointer) with explicit LXI H, <base address> initialization at the start of every sensor-read routine, and increment/reset the pointer deterministically when scanning multiple channels.
- Save critical values immediately using PUSH/POP or dedicated memory locations (STA) so the accumulator's sensor value is not accidentally overwritten by an intervening instruction before the compare (CMP) executes.

- Use direct addressing (STA/LDA) only for genuinely fixed control/status variables, with address constants defined once (e.g., via assembler EQU) rather than repeated magic numbers, to eliminate transcription errors.
- Use conditional jump instructions (JZ, JC, JNC) immediately after CMP to branch reliably on the comparison outcome, ensuring the actuator response always matches the actual, freshly read sensor value.

5. Conclusion

The occasional incorrect temperature readings in this 8085-based control system are best explained as addressing-mode misuse — most likely uninitialized/incorrect indirect pointers or stale direct-address references — rather than a sensor hardware fault. By disciplining the program to use register-indirect addressing with explicit pointer initialization for multi-channel sensor scanning, isolating the accumulator's sensor value immediately via STA, and pairing IN/OUT instructions with proper status checks, the instruction set of the 8085 can be used to guarantee that every temperature value processed by the actuator logic is accurate, current, and correctly sourced.

8085 Assembly

Code:

IN 00H ; Read temperature sensor

MOV B,A ; Store value

MVI A,32H ; Threshold = 50°C

CMP B

JC COOLER

JMP HEATER

COOLER:

MVI A,01H

OUT 01H

HLT

HEATER:

MVI A,00H

OUT 01H

HLT

Application: Reads temperature and controls heater/cooler.

Question 4: Segmentation in 8086 Microprocessor: Avoiding Data-Access and Stack Overflow Errors

Given Scenario: A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.

1. Understanding the Problem Context

The 8086 microprocessor uses a segmented memory model to address 1 MB of memory using 16-bit registers, by dividing memory into logical segments — Code Segment (CS), Data Segment (DS), Stack Segment (SS), and Extra Segment (ES) — each pointed to by a dedicated segment register, combined with a 16-bit offset to form a 20-bit physical address ($\text{Physical Address} = \text{Segment} \times 10\text{H} + \text{Offset}$). When multiple programs run concurrently, each should ideally operate within its own correctly bounded segments. The reported symptoms — incorrect data access and stack overflow — indicate that segment boundaries or segment-register values are being mismanaged, causing one program's memory activity to intrude on another's, or on its own code/stack region.

2. How Segmentation Works in the 8086

Code Segment (CS): Combined with the Instruction Pointer (IP), CS:IP forms the address of the next instruction to be fetched. All instruction fetches during a program's execution are relative to its CS value.

Data Segment (DS): Combined with an offset (often via a general-purpose register like BX or SI), DS gives the address of program data. Most direct and indirect data-addressing instructions default to DS unless a segment override prefix is used.

Stack Segment (SS): Combined with the Stack Pointer (SP), SS:SP addresses the top of the stack, used for PUSH/POP, CALL/RET, and interrupt handling. The stack grows downward (SP decreases on PUSH).

Extra Segment (ES): Used for additional data addressing, commonly with string instructions (MOVS, CMPS) alongside the DI register.

This model allows the same 16-bit offset value to refer to entirely different physical memory depending on which segment register is currently loaded, giving flexibility for multiprogramming — but it also means correctness fully depends on every segment register always holding the value appropriate to the currently executing program.

3. How Improper Segment Handling Causes the Reported Problems

- Incorrect data access: If DS is not correctly reloaded when switching between programs/tasks (context switch), a data instruction using an offset intended for Program B's data segment may execute while DS still points to Program A's segment, silently reading/writing the wrong physical memory.
- Segment overlap: Because segments in 8086 can overlap (segment start addresses are just multiples of 16 bytes, with no hardware-enforced isolation), two programs' logical segments may physically overlap if their base addresses and sizes aren't carefully allocated, causing one program to corrupt another's data or code.
- Stack overflow: If SS/SP are not correctly initialized per program, or if a program's stack segment is sized too small for its call/interrupt depth, repeated PUSH/CALL operations decrement SP past the bottom of the allocated stack segment, overwriting adjacent code or data segments — precisely the stack overflow behavior described.
- Missing segment-register save/restore during interrupts or task switches: if CS, DS, SS are not saved and restored around an interrupt service routine, execution can resume in the wrong segment context, corrupting subsequent instruction fetches or data access.

4. Managing Instruction Execution and Addressing Modes for Correctness

- Explicitly initialize all four segment registers (CS, DS, SS, ES) at the start of every task/program, and reload them (or use segment override prefixes such as ES: or SS: on the affected instruction) whenever addressing across segments is required, rather than relying on default DS association.
- Allocate non-overlapping segment base addresses with sufficient size margins for each concurrently running program, especially sizing the stack segment to comfortably exceed the program's maximum expected call/interrupt nesting depth.
- Use based and indexed addressing modes (e.g., [BX+SI], [BP+disp]) carefully — BP-based addressing defaults to the SS segment (useful for stack-frame-relative local variable access), while BX-based addressing defaults to DS; mixing these up is a frequent

source of the 'incorrect data access' fault, so the programmer must choose the addressing mode that matches the intended segment.

- Save and restore segment registers as part of the context-switch/interrupt routine (PUSH/POP the segment registers along with general-purpose registers) so each program resumes with exactly the segment context it had when interrupted.
- Add stack-pointer boundary checks (software guard values) in critical routines to detect an approaching stack overflow before SP crosses into adjacent segments.

5. Conclusion

Segmentation gives the 8086 the flexibility to run multiple programs by dividing memory into logical Code, Data, Stack, and Extra segments, but this flexibility is also its risk: because segments are not hardware-isolated, incorrect segment-register management directly causes wrong data access and stack overflows. Ensuring each program explicitly initializes and restores its own CS, DS, SS values, allocates adequately sized non-overlapping segments, and uses addressing modes consistent with the intended segment (BP→SS vs BX→DS) is essential to guarantee correct, isolated, and reliable execution of every concurrently running program.

8086 Assembly

Code:

MOV AX,DATA

MOV DS,AX

MOV AX,STACK

MOV SS,AX

MOV SP,100H

MOV AX,[2000H]

MOV BX,AX

Question 5: Number System Conversion Errors and Their Impact on Instruction Execution in Data Communication Systems

***Given Scenario:** A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.*

1. Understanding the Problem Context

Data communication systems represent packet headers, addresses, checksums, and payload fields in hexadecimal notation because it compactly represents the underlying binary bit patterns that the hardware and network protocols actually operate on (each hex digit maps exactly to 4 binary bits). The scenario describes a case where an engineer misinterprets a hexadecimal value during debugging or manual data entry, which propagates into the software as an incorrect binary value, ultimately causing the CPU to process and act on wrong data — an error that originates in a simple representational mistake but manifests as a functional/instructional fault.

2. Importance of Hexadecimal–Binary Conversion in Such Systems

- Compact, human-readable representation: A 32-bit IP address or a 16-bit checksum is far easier to read, communicate, and debug as 8 or 4 hex digits than as a long binary string, reducing (but not eliminating) human transcription error.
- Exact positional correspondence: Because $16 = 2^4$, each hex digit corresponds to exactly four binary bits with no rounding or ambiguity, so accurate hex-to-binary conversion is essential for protocol fields (flags, port numbers, TTL values, opcodes) to be interpreted with bit-level precision.
- Memory and register alignment: CPU registers, memory addresses, and instruction opcodes are naturally expressed in hex because they align with byte/word/nibble boundaries used internally by the processor, making hex the natural interface between human-level debugging and the machine's actual binary operation.

- Efficient encoding for transmission and storage: Representing packet fields in hex keeps debugging tools, log files, and packet captures compact while remaining an exact, lossless representation of the transmitted binary data.

3. How Conversion Errors Affect Instruction Execution

Once a hexadecimal value is misread or mis-converted (for example, confusing 0x1A (binary 00011010, decimal 26) with 0x1a as a different intended digit, or misplacing a nibble), the binary value loaded into a register or memory location no longer represents the actual packet field. Because CPU instructions operate purely on the binary pattern they are given — with no awareness of the 'intended' value — this error propagates directly into execution:

- Incorrect operand values: Arithmetic and logical instructions (ADD, CMP, AND) executed on the wrong binary operand produce wrong results, which may then drive an incorrect conditional branch (e.g., mis-evaluating a packet's protocol-type field and invoking the wrong handler).
- Address computation errors: If the misinterpreted hex value is used as part of a memory address or offset, the CPU will fetch/store data at the wrong memory location, potentially corrupting unrelated data or causing an incorrect data access fault.
- Checksum/validation failures or false-passes: An incorrectly converted checksum field can cause a valid packet to be wrongly rejected, or in the worst case a corrupted packet to be wrongly accepted, since the comparison instruction is comparing against a flawed reference value.
- Cascading CPI/performance impact: Misinterpreted control fields (e.g., packet length) can cause the CPU to read too many or too few bytes, triggering additional error-handling instructions, retries, or exception branches, which increases the effective instruction count and CPI for processing that packet.

4. Role of Efficient CPU Design and Benchmarking in Detecting and Minimizing Such Issues

CPU Design measures: Hardware-level parity/checksum verification units, dedicated CRC-computation instructions, and boundary/range-checking logic (e.g., memory protection units) can automatically flag a binary value that falls outside an expected valid range immediately after conversion, catching the error at execution time rather than allowing it to silently corrupt further processing. Well-designed instruction sets that provide direct hexadecimal-aware data movement

and comparison instructions also reduce the number of manual/software conversion steps where human or logic errors can be introduced.

Benchmarking measures: Running systematic benchmarks with known, verified hex/binary test vectors (unit tests comparing expected vs. actual converted values, fuzz-testing with edge-case packet fields) allows engineers to measure how often and under what conditions conversion-related instruction faults occur. Performance benchmarking that tracks CPI and instruction counts for packet-processing routines can also reveal anomalies — an unexpected rise in CPI for a particular packet type may indicate that conversion errors are triggering extra error-handling instructions, providing an early, measurable signal of the underlying data-interpretation bug.

Together, robust CPU-level validation hardware and disciplined benchmarking create both a preventive mechanism (catching invalid binary values before they influence execution) and a diagnostic mechanism (using performance metrics to detect when conversion errors are already degrading system behavior).

5. Conclusion

Hexadecimal representation exists purely as a convenience for humans; the CPU always operates on the underlying binary value, so any hex-to-binary conversion error becomes a genuine instruction-execution error — wrong operands, wrong addresses, or wrong branch decisions. Preventing this requires both careful, standardized conversion practices in software and hardware-assisted validation (parity/CRC checks, range checks) built into CPU design, supported by continuous benchmarking of packet-processing routines to detect abnormal CPI or error rates that reveal conversion faults before they cause serious data-processing failures in the real-time communication system.

Code:

```
#include <stdio.h>

int main() {

    int hexValue = 0x1A;

    printf("Hexadecimal : %X\n", hexValue);

    printf("Decimal    : %d\n", hexValue);

    return 0;

}
```

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand